



代码逐行解

读：`src/agents/system-prompt.ts`

本文件是 OpenClaw 的 **system prompt** 组装引擎——Agent 每次调用 LLM 前，都通过此文件构建完整的 system prompt 字符串。

全文 822 行，包含 2 个导出函数 + 15 个内部辅助函数 + 2 个类型 + 2 个常量。

零、TypeScript 速成（给 C++/Python 背景）

在逐行讲解之前，先补齐你需要的 TypeScript 基础。

0.1 TypeScript 是什么

TypeScript = JavaScript + 静态类型系统。编译后产出 JavaScript 运行。类比：

- C++ 编译为机器码 → TypeScript 编译为 JavaScript
- Python 有 type hints (`def foo(x: int) -> str`) → TypeScript 的类型是强制检查的，不只是标注

0.2 变量声明

```
const x = 42;           // 不可重新赋值（类似 C++ 的 const int x = 42;）
let y = "hello";        // 可重新赋值（类似 C++ 的 std::string y = "hello;"）
// 没有 var（过时了，别用）
```

Python 对比：Python 没有 const/let 区分，所有变量都可重赋值。

0.3 类型标注

```
// 基础类型
let name: string = "Alice";
let age: number = 30;           // 没有 int/float 之分, 统一是 number
let active: boolean = true;

// 数组
let items: string[] = ["a", "b"]; // 类似 Python 的 List[str]

// 函数
function add(a: number, b: number): number {
    return a + b;
}
// 等价箭头函数 (类似 Python 的 lambda, 但可以多行)
const add = (a: number, b: number): number => {
    return a + b;
};
// 极简箭头函数 (单表达式时省略 {} 和 return)
const add = (a: number, b: number): number => a + b;
```

0.4 interface 和 type

```
// interface — 定义对象形状 (类似 C++ 的纯虚类, Python 的 Protocol)
interface Person {
    name: string;
    age: number;
    email?: string; // ? 表示可选字段 (可以不传)
}

// type — 类型别名, 更灵活
type Status = "ok" | "error" | "pending"; // 联合类型 (类似 C++ 的 enum class)
type Result = { value: number } | { error: string }; // 联合类型 (类似 std::variant)
```

0.5 import / export

```
// 导出 (类似 Python 的 __all__ 或 C++ 的 public header)
export function foo() { ... }           // 命名导出
export type MyType = { ... };           // 导出类型
export default function bar() { ... }   // 默认导出 (每个文件只能一个)

// 导入
import { foo, MyType } from "./my-module.js"; // 命名导入
import type { MyType } from "./my-module.js"; // 仅导入类型 (编译后消失, 不产生运行时代码)
import bar from "./my-module.js";           // 导入默认导出
```

关键: `import type` 只导入类型信息, 不产生运行时 `import`。这对 `lazy loading` 很重要。

0.6 可选链和空值合并

```
// ?. — 可选链 (类似 Python 3.10+ 的 None-aware)
const name = user?.profile?.name; // 从左到右安全取值: 先看 user 在不在; 在的话再取 profile; pr
// C++ 没有等价物, Python 中需要 getattr(user, 'profile', None)

// ?? — 空值合并 (仅 null/undefined 时取右侧)
const port = config.port ?? 3000; // 如果 port 是 null 或 undefined, 用 3000
// 区别于 ||: 0 ?? 3000 → 0 (因为 0 不是 null), 0 || 3000 → 3000 (因为 0 是 falsy)
```

0.7 展开运算符 ...

```
// 数组展开 (类似 Python 的 *args 解包)
const a = [1, 2];
const b = [0, ...a, 3]; // [0, 1, 2, 3]

// 对象展开 (类似 Python 的 **kwargs 解包)
const base = { x: 1, y: 2 };
const extended = { ...base, z: 3 }; // { x: 1, y: 2, z: 3 }

// 在函数参数中用于"剩余参数"
function sum(...nums: number[]): number { ... }
```

0.8 模板字符串

```
const name = "Alice";
const greeting = `Hello, ${name}!`; // 反引号 `` ``，支持内嵌表达式
// 类似 Python 的 f"Hello, {name}!"
// C++ 没有原生等价物
```

0.9 三元运算符和条件表达式

```
// condition ? valueIfTrue : valueIfFalse
const label = count > 0 ? "has items" : "empty";
// 等价 Python: label = "has items" if count > 0 else "empty"

// 嵌套三元 (TypeScript 中常见, 注意缩进)
const result = a === true
  ? "yes"
  : b === true
    ? "maybe"
    : "no";
```

0.10 Map 和 Set

```
// Map — 类似 Python 的 dict, C++ 的 std::map
const m = new Map<string, number>();
m.set("a", 1);
m.get("a"); // 1
m.has("a"); // true

// Set — 类似 Python 的 set, C++ 的 std::unordered_set
const s = new Set<string>(["a", "b"]);
s.has("a"); // true
```

0.11 数组方法（高频使用）

```
const arr = [1, 2, 3, 4, 5];

// .map() — 转换每个元素（类似 Python 的 list comprehension [f(x) for x in arr]）
arr.map(x => x * 2);           // [2, 4, 6, 8, 10]

// .filter() — 筛选（类似 Python 的 [x for x in arr if x > 2]）
arr.filter(x => x > 2);        // [3, 4, 5]

// .some() — 任一满足（类似 Python 的 any()）
arr.some(x => x > 4);          // true

// .join() — 拼接为字符串（类似 Python 的 ",".join()）
["a", "b", "c"].join("|");    // "a|b|c"

// .toSorted() — 返回排序后的新数组（不修改原数组）
arr.toSorted((a, b) => b - a); // [5, 4, 3, 2, 1]

// Boolean 作为 filter — 过滤掉 falsy 值 (0, "", null, undefined, false)
["hello", "", null, "world"].filter(Boolean); // ["hello", "world"]
```

0.12 解构赋值

```
// 对象解构（类似 Python 3.10+ 的 match/case，但更常用）
const { level, channel } = params.reactionGuidance;
// 等价 Python: level = params.reactionGuidance["level"]; channel = ...

// 数组解构
const [first, second] = [10, 20];
```

0.13 类型断言 **as**

```
const result = someValue as Record<string, string>;
// 告诉编译器"我知道这个值的类型"，不产生运行时代码
// 类似 Python 的 cast() (from typing import cast) 或 C++ 的 static_cast<>
```

0.14 Record 和 Partial

```
// Record<K, V> — 键为 K 类型、值为 V 类型的对象
// 类似 Python 的 Dict[str, int], C++ 的 std::map<string, int>
type Config = Record<string, number>;

// Partial<T> — T 的所有字段变为可选
// 类似 Python 的 TypedDict(total=False)
type PartialConfig = Partial<Config>; // 所有字段可选
```

0.15 Object.entries / Object.fromEntries

```
const obj = { a: 1, b: 2 };

// Object.entries() → 键值对数组 (类似 Python 的 dict.items())
Object.entries(obj); // [["a", 1], ["b", 2]]

// Object.fromEntries() → 从键值对数组构建对象 (类似 Python 的 dict())
Object.fromEntries([["a", 1], ["b", 2]]); // { a: 1, b: 2 }
```

现在你具备了阅读本文件所需的全部 TypeScript 知识。下面开始逐块解读。

一、导入区 (L1-25)

```
import { createHmac, createHash } from "node:crypto"; // L1
import type { ReasoningLevel, ThinkLevel } from "../auto-reply/thinking.js"; // L2
import { SILENT_REPLY_TOKEN } from "../auto-reply/tokens.js"; // L3
// ... 更多导入
```

逐行解释

行	导入	说明
L1	<code>createHmac, createHash</code>	Node.js 内置加密模块。用于将 owner ID 哈希化（隐私保护）
L2	<code>type ReasoningLevel, ThinkLevel</code>	仅类型导入——推理级别的类型定义（"off"/"on"/"s"等）
L3	<code>SILENT_REPLY_TOKEN</code>	常量字符串 <code>__SILENT__</code> ，Agent 用此标记"无需回复用户"
L4	<code>resolveChannelApprovalCapability</code>	查询某通道是否支持原生审批 UI（如按钮）
L5	<code>getChannelPlugin</code>	根据通道名获取通道插件实例
L6	<code>type MemoryCitationsMode</code>	记忆引用模式的类型
L7	<code>buildMemoryPromptSection</code>	构建记忆插件的 prompt section
L8-11	字符串归一化工具	小写化、去空格等
L12	<code>listDeliverableMessageChannels</code>	获取所有可发送消息的通道列表（用于 prompt 中的 channel 选项）
L13	<code>type ResolvedTimeFormat</code>	时间格式类型
L14	<code>type EmbeddedContextFile</code>	上下文文件结构 <code>{ path: string, content: string }</code>
L15	<code>type EmbeddedSandboxInfo</code>	沙盒信息类型
L16-19	prompt cache 稳定性工具	确保 prompt 字节确定性的辅助函数
L20	<code>sanitizeForPromptLiteral</code>	清理字符串中的 prompt injection 风险字符
L21	<code>SYSTEM_PROMPT_CACHE_BOUNDARY</code>	cache 分界线标记字符串
L22-25	Provider 贡献类型	Provider 可以覆盖 prompt 的特定 section

设计洞察：

- `import type` 只导入类型（编译时用），不产生运行时 import → 减少启动开销

- 导入来自 `../shared/`、`../utils/`、`../channels/` 等不同模块 → 体现了 **system prompt** 是跨模块信息的汇聚点
-

二、类型定义 (L27-34)

```
export type PromptMode = "full" | "minimal" | "none";    // L33
type OwnerIdDisplay = "raw" | "hash";                    // L34
```

PromptMode（导出）

控制 **system prompt** 包含哪些 **section**：

- `"full"`：主 Agent，包含全部 **section**
- `"minimal"`：Subagent，只保留 Tooling + Workspace + Runtime（减少 token 消耗）
- `"none"`：只返回一行 **identity** 字符串

C++ 类比：`enum class PromptMode { Full, Minimal, None };`

Python 类比：`PromptMode = Literal["full", "minimal", "none"]`

OwnerIdDisplay（内部）

控制 **owner** 信息在 **prompt** 中的显示方式：

- `"raw"`：直接显示（如手机号）
 - `"hash"`：显示哈希摘要（隐私保护）
-

三、上下文文件排序常量 (L36-46)

```
const CONTEXT_FILE_ORDER = new Map<string, number>([ // L36-44
  ["agents.md", 10],
  ["soul.md", 20],
  ["identity.md", 30],
  ["user.md", 40],
  ["tools.md", 50],
  ["bootstrap.md", 60],
  ["memory.md", 70],
]);

const DYNAMIC_CONTEXT_FILE_BASENAMES = new Set(["heartbeat.md"]); // L46
```

作用

`CONTEXT_FILE_ORDER` 定义了上下文文件在 `prompt` 中的确定性排序。数值越小越靠前。

为什么需要确定性排序？ Prompt caching。如果文件顺序每次不同，即使内容一样，API 端的 KV cache 也会 miss（因为字节序列变了）。

`DYNAMIC_CONTEXT_FILE_BASENAMES` 标记哪些文件是“频繁变化”的（如 `heartbeat.md`），这些文件会被放到 `cache boundary` 之下，避免每次更新都导致整个 `static zone` 的 cache 失效。

C++ 类比： `static const std::map<std::string, int> CONTEXT_FILE_ORDER = {...};`

四、上下文文件辅助函数 (L48-77)

`normalizeContextFilePath(pathValue)` (L48-50)

```
function normalizeContextFilePath(pathValue: string): string {
  return pathValue.trim().replace(/\\/g, "/");
}
```

去首尾空格 + 反斜杠转正斜杠（Windows 兼容）。

- `trim()` → Python 的 `strip()`
- `.replace(/\\/g, "/")` → 正则替换，`/\\/g` 是正则字面量，`\\` 匹配反斜杠，`g` 表示全局替换
- Python 等价： `path.strip().replace("\\", "/")`

getContextFileBasename(pathValue) (L52-55)

```
function getContextFileBasename(pathValue: string): string {  
    const normalizedPath = normalizeContextFilePath(pathValue);  
    return normalizeLowercaseStringOrEmpty(normalizedPath.split("/").pop() ?? normalizedPath)  
}
```

提取文件名并小写化。

- `.split("/")` → 按 `/` 分割为数组
- `.pop()` → 取最后一个元素（即文件名）。注意 `pop()` 可能返回 `undefined`（空数组时）
- `?? normalizedPath` → 如果 `pop()` 返回 `undefined`，则用完整路径
- Python 等价： `os.path.basename(path).lower()`

isDynamicContextFile(pathValue) (L57-59)

```
function isDynamicContextFile(pathValue: string): boolean {  
    return DYNAMIC_CONTEXT_FILE_BASENAMES.has(getContextFileBasename(pathValue));  
}
```

检查文件是否属于动态文件（当前只有 `heartbeat.md`）。

sortContextFilesForPrompt(contextFiles) (L61-77)

```
function sortContextFilesForPrompt(contextFiles: EmbeddedContextFile[]): EmbeddedContextFile[] {
  return contextFiles.toSorted((a, b) => {
    // a, b 是 toSorted 每次取出的两个待比较元素
    const aPath = normalizeContextFilePath(a.path); // 从元素 a 提取路径
    const bPath = normalizeContextFilePath(b.path);
    const aBase = getContextFileBasename(a.path); // 从元素 a 提取文件名
    const bBase = getContextFileBasename(b.path);
    const aOrder = CONTEXT_FILE_ORDER.get(aBase) ?? Number.MAX_SAFE_INTEGER;
    // ↑ 从 Map 查优先级 (如 "soul.md"→20), 查不到给最大值排最后
    const bOrder = CONTEXT_FILE_ORDER.get(bBase) ?? Number.MAX_SAFE_INTEGER;
    if (aOrder !== bOrder) return aOrder - bOrder; // 负数→a在前, 正数→b在前
    if (aBase !== bBase) return aBase.localeCompare(bBase); // 同优先级按文件名字母序
    return aPath.localeCompare(bPath); // 同文件名按完整路径字母序
  });
}
```

三级排序：

1. 优先级：CONTEXT_FILE_ORDER 中定义的（agents.md=10 最先，memory.md=70 最后）
2. 文件名字母序：未在 CONTEXT_FILE_ORDER 中的文件（优先级为 MAX_SAFE_INTEGER）按名称排
3. 完整路径字母序：同名文件按路径排

`.toSorted()` 是 ES2023 的方法，返回新数组不修改原数组（类似 Python 的 `sorted()` vs `.sort()`）。

设计洞察：`?? Number.MAX_SAFE_INTEGER` 让未配置优先级的文件排到最后。`Number.MAX_SAFE_INTEGER` 是 JS 中最大安全整数（ $2^{53} - 1$ ）。

五、 `buildProjectContextSection()` (L79-109)

```
function buildProjectContextSection(params: {
  files: EmbeddedContextFile[];
  heading: string;
  dynamic: boolean;
}) {
  if (params.files.length === 0) return [];
  const lines = [params.heading, ""];
  if (params.dynamic) {
    lines.push("The following frequently-changing project context files are kept below the");
  } else {
    const hasSoulFile = params.files.some(
      (file) => getContextFileBasename(file.path) === "soul.md",
    );
    lines.push("The following project context files have been loaded:");
    if (hasSoulFile) {
      lines.push("If SOUL.md is present, embody its persona and tone. ...");
    }
    lines.push("");
  }
  for (const file of params.files) {
    lines.push(`## ${file.path}`, "", file.content, "");
  }
  return lines;
}
```

参数

- `files` : 上下文文件列表
- `heading` : **section** 标题 (`"# Project Context"` 或 `"# Dynamic Project Context"`)
- `dynamic` : 是否是动态区

逻辑

1. 空文件列表 → 返回空数组 (不生成任何 **prompt** 内容)
2. 动态区 → 添加说明"这些是频繁变化的文件"

3. 静态区 → 添加说明"这些文件已加载"。如果包含 `soul.md`，额外添加 `persona` 指导
4. 每个文件以 `## 文件路径` 为标题，后跟文件内容

设计洞察：`soul.md` 触发特殊的人设指导——这是让 **Agent** 具备"性格"的机制。如果用户在工作目录放了 `soul.md`，**Agent** 会被指示"遵循其语气和人设"。

六、 `buildSkillsSection()` (L111-127)

```
function buildSkillsSection(params: { skillsPrompt?: string; readToolName: string }) {
  const trimmed = params.skillsPrompt?.trim();
  if (!trimmed) return [];
  return [
    "## Skills (mandatory)",
    "Before replying: scan <available_skills> <description> entries.",
    "- If exactly one skill clearly applies: read its SKILL.md at <location> with `${params.readToolName}`,",
    "- If multiple could apply: choose the most specific one, then read/follow it.",
    "- If none clearly apply: do not read any SKILL.md.",
    "Constraints: never read more than one skill up front; only read after selecting.",
    "- When a skill drives external API writes, assume rate limits: ...",
    trimmed, // 实际的 <available_skills> XML 内容
    "",
  ];
}
```

作用

构建 **Skills section**。注意 `params.readToolName` 是动态的（不同环境下读文件的工具名可能不同，如 `read` 或 `Read`）。

关键规则注入：

- 必须先扫描再决定用哪个 **skill**
- 最多读一个 **SKILL.md**（避免 token 浪费）
- 外部 **API** 写入要注意限流

`trimmed` 就是之前 `resolveSkillsPromptForRun()` 生成的 `<available_skills>` XML。

七、buildMemorySection() (L129-142)

```
function buildMemorySection(params: {
  isMinimal: boolean;
  includeMemorySection?: boolean;
  availableTools: Set<string>;
  citationsMode?: MemoryCitationsMode;
}) {
  if (params.isMinimal || params.includeMemorySection === false) return [];
  return buildMemoryPromptSection({
    availableTools: params.availableTools,
    citationsMode: params.citationsMode,
  });
}
```

门卫函数：minimal 模式或显式禁用时跳过。实际构建委托给

buildMemoryPromptSection()（来自 plugins/memory-state.ts）。

八、Owner Identity 相关函数 (L144-173)

buildUserIdentitySection() (L144-149)

```
function buildUserIdentitySection(ownerLine: string | undefined, isMinimal: boolean) {
  if (!ownerLine || isMinimal) return [];
  return ["## Authorized Senders", ownerLine, ""];
}
```

生成授权用户 section。!ownerLine 在 JS 中，undefined、null、""（空字符串）都是 falsy。

formatOwnerDisplayId() (L151-157)

```
function formatOwnerDisplayId(ownerId: string, ownerDisplaySecret?: string) {
  const hasSecret = ownerDisplaySecret?.trim();
  const digest = hasSecret
    ? createHmac("sha256", hasSecret).update(ownerId).digest("hex")
    : createHash("sha256").update(ownerId).digest("hex");
  return digest.slice(0, 12);
}
```

将 owner ID（如手机号）转为 12 字符的哈希摘要。

- 有 secret → 用 HMAC-SHA256（keyed hash，更安全，防彩虹表）
- 无 secret → 用 SHA-256（plain hash）
- `.slice(0, 12)` → 取前 12 个 hex 字符（48 bit）

C++ 等价思路： `std::string digest = sha256(ownerId).substr(0, 12);`

设计洞察：prompt 中不直接暴露用户手机号/ID，而是显示哈希摘要。这是隐私设计——即使 prompt 被泄露，也无法反推出真实身份。

buildOwnerIdentityLine() (L159-173)

```
function buildOwnerIdentityLine(
  ownerNumbers: string[],
  ownerDisplay: OwnerIdDisplay,
  ownerDisplaySecret?: string,
) {
  const normalized = ownerNumbers.map((value) => value.trim()).filter(Boolean);
  if (normalized.length === 0) return undefined;
  const displayOwnerNumbers =
    ownerDisplay === "hash"
      ? normalized.map((ownerId) => formatOwnerDisplayId(ownerId, ownerDisplaySecret))
      : normalized;
  return `Authorized senders: ${displayOwnerNumbers.join(", ")}. These senders are allowli
}
```

将 owner 列表格式化为一行文本。根据 `ownerDisplay` 模式决定显示原始值还是哈希。

九、简单 Section 构造函数 (L175-279)

buildTimeSection() (L175-180)

```
function buildTimeSection(params: { userTimezone?: string }) {  
  if (!params.userTimezone) return [];  
  return ["## Current Date & Time", `Time zone: ${params.userTimezone}`, ""];  
}
```

有时区配置则生成时间 section，否则返回空。

buildReplyTagsSection() (L182-196)

生成回复标签语法说明。教 Agent 如何使用 `[[reply_to_current]]` 等标签实现消息引用。

- `isMinimal` 时跳过 (subagent 不需要回复标签)

buildMessagingSection() (L198-236)

消息路由和消息工具使用指导。关键逻辑：

- `params.availableTools.has("message")` → 如果有 message 工具，追加其使用指南
- `params.inlineButtonsEnabled` → 如果通道支持内联按钮，添加按钮指导
- `params.messageToolHints` → 通道特定的额外提示

buildVoiceSection() (L238-247)

TTS (文本转语音) 指导。只在非 minimal 模式且有 TTS 配置时生成。

buildDocsSection() (L249-265)

文档链接 section。包含 docs 路径、GitHub、Discord、ClawhHub 链接。

buildExecutionBiasSection() (L267-279)

执行偏好指导。核心规则：用户让你做事，马上开始做，别只说不做。

```
"If the user asks you to do the work, start doing it in the same turn."
```

```
"Commentary-only turns are incomplete when tools are available and the next action is clear."
```

这是 harness 工程中非常关键的 prompt 设计——它解决了 LLM 的一个常见问题：倾向于"说说我会做什么"而不是真正去做。

十、Provider Override 机制 (L281-315)

normalizeProviderPromptBlock() (L281-287)

```
function normalizeProviderPromptBlock(value?: string): string | undefined {  
  if (typeof value !== "string") return undefined;  
  const normalized = normalizeStructuredPromptSection(value);  
  return normalized || undefined;  
}
```

清理 provider 提供的 prompt 块。空字符串转为 `undefined`。

buildOverridablePromptSection() (L289-298)

```
function buildOverridablePromptSection(params: {  
  override?: string;  
  fallback: string[];  
}): string[] {  
  const override = normalizeProviderPromptBlock(params.override);  
  if (override) return [override, ""];  
  return params.fallback;  
}
```

这是一个关键设计模式：Provider 可以替换 prompt 的特定 section。

- 有 `override` → 使用 `provider` 提供的内容
- 无 `override` → 使用 `OpenClaw` 的默认内容 (`fallback`)

例如 `Anthropic provider` 可能想要不同的 "Tool Call Style", 就通过

`promptContribution.sectionOverrides.tool_call_style` 传入自定义文本。

C++ 类比: 虚函数重写 — `base class` 提供默认实现, `derived class` 可以 `override`。

`buildExecApprovalPromptGuidance()` (L300-315)

根据当前通道是否支持原生审批 UI (如 `WebChat` 的按钮), 生成不同的审批指导:

- 支持原生 UI → "依赖审批卡片/按钮, 不要发纯文本 `/approve`"
- 不支持 → "发送 `/approve` 命令文本给用户"

十一、主函数 `buildAgentSystemPrompt()` (L317-777)

这是整个文件的核心, 460 行。我按逻辑块拆解。

11.1 函数签名 (L317-366)

```
export function buildAgentSystemPrompt (params: {
  workspaceDir: string;
  defaultThinkLevel?: ThinkLevel;
  reasoningLevel?: ReasoningLevel;
  extraSystemPrompt?: string;
  ownerNumbers?: string[];
  // ... 30+ 个参数
}) {
```

参数用一个匿名对象类型 (内联 `interface`) 声明。**30+** 个参数体现了 `system prompt` 需要汇聚多少信息。

关键参数:

参数	类型	说明
<code>workspaceDir</code>	string	工作目录路径
<code>promptMode</code>	PromptMode	full/minimal/none
<code>toolNames</code>	string[]	可用工具列表
<code>contextFiles</code>	EmbeddedContextFile[]	agents.md , soul.md 等
<code>skillsPrompt</code>	string	<code><available_skills></code> XML
<code>heartbeatPrompt</code>	string	心跳轮询 prompt
<code>runtimeInfo</code>	object	agent/OS/model/channel 等运行时信息
<code>sandboxInfo</code>	EmbeddedSandboxInfo	沙盒配置
<code>promptContribution</code>	ProviderSystemPromptContribution	Provider 的 prompt 贡献
<code>ownerNumbers</code>	string[]	授权用户列表
<code>reactionGuidance</code>	object	emoji 反应策略
<code>extraSystemPrompt</code>	string	额外上下文（群聊等）

11.2 工具名解析 (L367-392)

```
const acpEnabled = params.acpEnabled !== false;
const rawToolNames = (params.toolNames ?? []).map((tool) => tool.trim());
const canonicalToolNames = rawToolNames.filter(Boolean);

// 去重但保留原始大小写
const canonicalByNormalized = new Map<string, string>();
for (const name of canonicalToolNames) {
  const normalized = normalizeLowercaseStringOrEmpty(name);
  if (!canonicalByNormalized.has(normalized)) {
    canonicalByNormalized.set(normalized, name);
  }
}

const resolveToolName = (normalized: string) =>
  canonicalByNormalized.get(normalized) ?? normalized;
```

为什么需要这么复杂的工具名处理？

1. 工具名是大小写敏感的（`Read` vs `read`）
2. 但内部查找需要归一化比较
3. `canonicalByNormalized` Map：小写名 → 原始大小写名
4. `resolveToolName()`：给定小写名，返回原始大小写名（在 `prompt` 中使用正确的大小写）

```
const availableTools = new Set(normalizedTools);
const hasSessionsSpawn = availableTools.has("sessions_spawn");
const hasUpdatePlanTool = availableTools.has("update_plan");
const hasCronTool = availableTools.has("cron") || canonicalToolNames.length === 0;
const readToolName = resolveToolName("read");
const execToolName = resolveToolName("exec");
```

feature flag 模式：根据可用工具决定 `prompt` 中包含哪些指导。例如：

- 有 `cron` 工具 → 指导"用 `cron` 做定时任务"
- 没有 `cron` → 指导"用 `sleep/poll` 等待"
- 有 `update_plan` → 指导"用 `update_plan` 跟踪多步任务"

11.3 Provider 贡献解析 (L397-407)

```
const providerStablePrefix = normalizeProviderPromptBlock(promptContribution?.stablePrefix);
const providerDynamicSuffix = normalizeProviderPromptBlock(promptContribution?.dynamicSuffix);
const providerSectionOverrides = Object.fromEntries(
  Object.entries(promptContribution?.sectionOverrides ?? {})
    .map(([key, value]) => [key, normalizeProviderPromptBlock(...)])
    .filter(([, value]) => Boolean(value)),
) as Partial<Record<ProviderSystemPromptSectionId, string>>;
```

Provider（如 Anthropic、OpenAI）可以贡献三种 prompt 内容：

1. `stablePrefix`：静态前缀（放在 `cache boundary` 之上）
2. `dynamicSuffix`：动态后缀（放在 `cache boundary` 之下）
3. `sectionOverrides`：替换特定 `section` 的内容

`Object.entries()` → `map()` → `filter()` → `Object.fromEntries()` 这个链式操作是 TypeScript 中常见的"transform + filter object"模式：

1. 对象 → 键值对数组
2. 转换每个键值对
3. 过滤掉无效的
4. 键值对数组 → 对象

Python 等价：`{k: normalize(v) for k, v in overrides.items() if normalize(v)}`

11.4 其他预计算 (L408-491)

```
const ownerDisplay = params.ownerDisplay === "hash" ? "hash" : "raw";
const ownerLine = buildOwnerIdentityLine(params.ownerNumbers ?? [], ownerDisplay, ...);
const reasoningHint = params.reasoningTagHint ? [...].join(" ") : undefined;
const reasoningLevel = params.reasoningLevel ?? "off";
const userTimezone = params.userTimezone?.trim();
const skillsPrompt = ...;
const heartbeatPrompt = ...;
// ... 各种 section 预构建
```

这里做了大量预计算，将所有参数归一化、`section` 预构建。注意：

```
if (promptMode === "none") {
  return "You are a personal assistant operating inside OpenClaw.";
}
```

none 模式的早期返回——只需一行 **identity**，跳过后续所有复杂逻辑。

11.5 主 prompt 数组组装 (L493-670)

```
const lines = [
  "You are a personal assistant operating inside OpenClaw.",
  "",
  "## Tooling",
  ...
];
```

整个 **prompt** 被组装为一个 `string[]`（字符串数组），最后通过 `lines.filter(Boolean).join("\n")` 拼接。

为什么用数组而不是字符串拼接？

1. 条件 **section** 可以用 `...spread` 展开空数组 `[]` → 不产生任何行
2. 条件行可以用三元表达式返回 `""` → 最后被 `filter(Boolean)` 过滤掉
3. 更容易维护和阅读

关键 **spread** 模式：

```
...(hasCronTool
  ? [
    "用 cron 做定时任务...",
    "exec/process 只用于...",
    ...
  ]
  : [
    "用 sleep 等待...",
    ...
  ]),
```

C++ 没有等价语法。Python 类比：

```
lines = [  
    "identity",  
    *(cron_lines if has_cron else poll_lines), # Python 3.5+ 解包  
]
```

11.6 Overridable Section 组装 (L531-563)

```
...buildOverridablePromptSection({  
    override: providerSectionOverrides.interaction_style,  
    fallback: [],  
}),  
...buildOverridablePromptSection({  
    override: providerSectionOverrides.tool_call_style,  
    fallback: [  
        "## Tool Call Style",  
        "Default: do not narrate routine, low-risk tool calls ...",  
        ...  
    ],  
}),  
...buildOverridablePromptSection({  
    override: providerSectionOverrides.execution_bias,  
    fallback: buildExecutionBiasSection({ isMinimal }),  
}),  
...buildOverridablePromptSection({  
    override: providerStablePrefix,  
    fallback: [],  
}),
```

四个可被 provider 覆盖的位置。如果 provider 不提供 override，则使用 fallback 默认内容。如果 fallback 也是 []（如 interaction_style），则该位置不产生任何内容。

11.7 Sandbox Section (L605-652)

最复杂的条件 section 之一。沙盒启用时，生成大量配置说明：

- 容器工作目录

- 宿主挂载源
- workspace 访问级别
- 浏览器桥接
- noVNC 观察者
- 宿主浏览器控制
- elevated exec 权限

```
params.sandboxInfo?.enabled
  ? [
    "You are running in a sandboxed runtime ...",
    // 10+ 条件行
  ]
  .filter(Boolean)      // 过滤掉空字符串
  .join("\n")           // 拼接为单个字符串
  : "",
```

注意这里用了 `.filter(Boolean).join("\n")` 把多行压成一个字符串——因为这整个 sandbox block 在外层数组中是一个单元素。

11.8 后续 Section (L653-670)

```
...buildUserIdentitySection(ownerLine, isMinimal),
...buildTimeSection({ userTimezone }),
"## Workspace Files (injected)",
...buildReplyTagsSection(isMinimal),
...buildMessagingSection({ ... }),
...buildVoiceSection({ ... }),
```

这些 section 的 spread 模式一致：builder 函数返回 `string[]` 或 `[]`（空数组），用 `...` 展开。

11.9 可选尾部 Section (L672-697)

```
if (params.reactionGuidance) {
  const { level, channel } = params.reactionGuidance; // 解构赋值
  const guidanceText =
    level === "minimal"
      ? [...].join("\n")
      : [...].join("\n");
  lines.push("## Reactions", guidanceText, "");
}
if (reasoningHint) {
  lines.push("## Reasoning Format", reasoningHint, "");
}
```

这些是用 `if + lines.push()` 而非 `spread` 模式添加的。两种写法效果相同，`push` 在有复杂逻辑时更清晰。

11.10 Context Files 分离 (L699-712)

```
const contextFiles = params.contextFiles ?? [];
const validContextFiles = contextFiles.filter(
  (file) => typeof file.path === "string" && file.path.trim().length > 0,
);
const orderedContextFiles = sortContextFilesForPrompt(validContextFiles);
const stableContextFiles = orderedContextFiles.filter((file) => !isDynamicContextFile(file));
const dynamicContextFiles = orderedContextFiles.filter((file) => isDynamicContextFile(file));
```

分离逻辑：

1. 过滤无效文件（path 为空）
2. 排序（确定性排序，见第四节）
3. 拆分为 `stable` 和 `dynamic` 两组

`stable` → cache boundary 之上 → 可被 prompt caching 复用

`dynamic` → cache boundary 之下 → 每轮可能变化

11.11 Silent Replies Section (L714-732)

```
if (!isMinimal) {
  lines.push(
    "## Silent Replies",
    `Use ${SILENT_REPLY_TOKEN} ONLY when no user-visible reply is required.` ,
    ...
    `✗ Wrong: "Here's help... ${SILENT_REPLY_TOKEN}"`,
    `✓ Right: ${SILENT_REPLY_TOKEN}`,
    "",
  );
}
```

教 Agent 何时可以“沉默”（不回复用户）。包含正确/错误示例——这是 prompt engineering 中的 **few-shot example** 技巧。

11.12 CACHE BOUNDARY (L734-737)

```
// Keep large stable prompt context above this seam so Anthropic-family
// transports can reuse it across labs and turns.
lines.push(SYSTEM_PROMPT_CACHE_BOUNDARY);
```

这一行是整个文件最重要的架构决策之一。

上面所有内容（L493-732）构成 **static zone** → 可被 prompt caching 复用。

下面的内容构成 **dynamic zone** → 每轮可能不同。

11.13 Dynamic Zone (L739-776)

```
// 动态上下文文件（如 heartbeat.md）
lines.push(
  ...buildProjectContextSection({
    files: dynamicContextFiles,
    heading: stableContextFiles.length > 0 ? "# Dynamic Project Context" : "# Project Context",
    dynamic: true,
  }),
);

// 群聊/session 额外上下文
if (extraSystemPrompt) {
  const contextHeader = promptMode === "minimal" ? "## Subagent Context" : "## Group Chat Context";
  lines.push(contextHeader, extraSystemPrompt, "");
}

// Provider 动态后缀
if (providerDynamicSuffix) {
  lines.push(providerDynamicSuffix, "");
}

// 心跳指导
if (!isMinimal && heartbeatPrompt) {
  lines.push("## Heartbeats", ...);
}

// Runtime 信息（始终存在）
lines.push(
  "## Runtime",
  buildRuntimeLine(runtimeInfo, runtimeChannel, runtimeCapabilities, params.defaultThinkLevel,
    `Reasoning: ${reasoningLevel} (hidden unless on/stream). ...`,
  ),
);

// 最终拼接
return lines.filter(Boolean).join("\n");
```

```
lines.filter(Boolean).join("\n") :
```

1. `filter(Boolean)` 过滤掉所有 `falsey` 值 (`""`、`undefined`、`null`、`false`)
2. `join("\n")` 用换行符拼接

这就是最终返回给调用方的 `system prompt` 字符串。

十二、`buildRuntimeLine()` (L779-821)

```
export function buildRuntimeLine(
  runtimeInfo?: { agentId?: string; host?: string; os?: string; ... },
  runtimeChannel?: string,
  runtimeCapabilities: string[] = [],
  defaultThinkLevel?: ThinkLevel,
): string {
  const normalizedRuntimeCapabilities = normalizePromptCapabilityIds(runtimeCapabilities);
  return `Runtime: ${[
    runtimeInfo?.agentId ? `agent=${runtimeInfo.agentId}` : "",
    runtimeInfo?.host ? `host=${runtimeInfo.host}` : "",
    runtimeInfo?.os
      ? `os=${runtimeInfo.os}${runtimeInfo?.arch ? ` (${runtimeInfo.arch})` : ""}`
      : runtimeInfo?.arch ? `arch=${runtimeInfo.arch}` : "",
    runtimeInfo?.model ? `model=${runtimeInfo.model}` : "",
    runtimeChannel ? `channel=${runtimeChannel}` : "",
    runtimeChannel
      ? `capabilities=${normalizedRuntimeCapabilities.length > 0
        ? normalizedRuntimeCapabilities.join(",") : "none"}`
      : "",
    `thinking=${defaultThinkLevel ?? "off"}`,
  ]
    .filter(Boolean)
    .join(" | ")}`;
}
```

生成类似这样的单行字符串：

```
Runtime: agent=main | host=mypc | os=linux (x64) | model=claude-sonnet-4-6 | channel=feish
```

构建方式：

1. 数组中每个元素是一个 `key=value` 片段（有值时）或 `""`（无值时）
2. `filter(Boolean)` 移除空字符串
3. `join(" | ")` 用管道符连接

`normalizePromptCapabilityIds()` 确保 capabilities 列表的排序确定性 → prompt caching 稳定性。

十三、全文架构总结

system-prompt.ts 的数据流：

params (30+ 参数)

↓

预计算层 (L367-491)

- └ 工具名归一化 + feature flag
- └ Provider 贡献解析
- └ Owner 身份构建
- └ 各 section 预构建

↓

组装层 (L493-776)

- └ lines = string[]
- └ Static Zone (16 个始终 section + 10 个条件 section)
- └ CACHE BOUNDARY
- └ Dynamic Zone (context files, group chat, heartbeat, runtime)

↓

输出层 (L776)

- └ lines.filter(Boolean).join("\n") → 完整 system prompt 字符串

核心设计模式回顾

1. 数组 + **spread** + **filter(Boolean)** + **join**：TypeScript 中构建动态字符串的惯用模式
2. **Overridable section**：provider 可替换默认 prompt 段落
3. **Cache boundary**：静态/动态分离，优化 API 端 KV cache
4. 确定性排序：文件、capabilities、工具名都经过排序处理

5. **Feature flag by tool availability** : 根据可用工具动态调整指导内容
6. 隐私 **by design** : owner ID 可哈希化
7. 早期返回 : `none` 模式立即返回，避免无谓计算